

DWG to ESP300

레이저 스테이지 랩 리그

DWG/DXF 도면을 ESP300 3축 모션 컨트롤러 경로로 변환하고, 브라우저에서 실기계 조그·UVC 카메라 기반 Z 자동초점·가공 실행까지 한 화면에서 다루는 시스템의 전체 운영 매뉴얼.

문서 버전 2026-09-09	대상 리그 dwg-to-esp300-lab-code	서버 포트 127.0.0.1:8768
모션 컨트롤러 Newport ESP300 · COM5 · 19200bps	경로 C:\Users\bjh14\dwg-to-esp300-lab-code	카메라 UVC, index 1, CAP_MSMF

목차

01 시스템 개요와 아키텍처

02 시작하기 — 서버 실행과 첫 연결

03 화면 구성 — Toolbar & Planner

04 화면 구성 — Manual Jog 패널

05 화면 구성 — Focus Camera & Autofocus 패널

06 자동초점 알고리즘 상세

07 가공 실행 — Run on Machine

08 HTTP API 레퍼런스

09 Standalone 스크립트 레퍼런스

10 자동 수집 데이터 & 로그

11 안전장치 전체 목록

A 부록 A — 트러블슈팅

B 부록 B — 튜닝 상수 전체 표

01 시스템 개요와 아키텍처

두 개의 독립된 코드 경로가 이 리그를 이룬다 — 하나는 도면을 오프라인 스크립트로 굽는 컴파일러, 하나는 실기계를 브라우저에서 직접 조작하는 서버.

1.1 전체 파이프라인

25mm 안팎의 작은 부품 도면(DWG/DXF)을 고정빔·이동유리 방식 레이저 스테이지의 이동 명령으로 변환하는 것이 목적. 두 경로:

- `dwg_to_esp300.py` — standalone CLI 변환기. `ezdxf` 로 도면을 파싱하고 필터링·회전·스케일·정렬한 뒤, ESP300 네이티브 ASCII 명령어(`PA`, `PR`, `VA`, `MO`, `MF`, `TP`)로 스테이지를 직접 구동하는 **완전한 standalone .py 스크립트를 f-string 템플릿으로 생성**한다. G-code나 명령 리스트 파일이 아니라 진짜 실행 가능한 파이썬 코드 한 벌을 뱉아낸다는 점이 핵심. Jupyter 노트북 버전과 SVG 미리보기도 같이 만들 수 있음.
- `visualizer_server.py` — `ThreadingHTTPServer` 기반 로컬 웹 앱. `dwg_to_esp300.py` 의 함수들(`LaserPath`, `apply_fit_scale`, `cap_scale_to_circle`, `extract_paths`, `generate_esp300_script` 등)을 그대로 import해서 브라우저 UI로 계획(planning)과 실기계 라이브 구동(조그, 카메라 초점 보정, "Run on machine")을 동시에 제공.

1.2 `visualizer_server.py` 핵심 클래스 4개

클래스	역할
<code>JogController</code>	ESP300 시리얼 통신 전체 — 연결/해제, 축별 <code>move()</code> , 절대좌표 <code>goto()</code> , <code>home()</code> / <code>set_local_home()</code> , E-STOP, 백그라운드 스레드로 전체 경로를 실행하는 <code>run_plan()</code> / <code>_run_worker()</code> .
<code>CameraController</code>	백그라운드 UVC 캡처 스레드(OpenCV). MJPEG 스트림과 blob 밝기 상태를 동시에 제공. <code>locate_bright_regions()</code> 로 red-glow/saturated-core 스팟 추적.
<code>FocusScanner</code>	Z 자동초점 스캔 오케스트레이션. <code>focus_autocal.scan_focus_z()</code> 를 호출하되 <code>move_z</code> 는 <code>JOG.move("z",...)</code> 에, <code>read_brightness</code> 는 카메라 blob score에 직접(HTTP 왕복 없이 in-process로) 연결.
<code>VisualizerHandler</code>	<code>do_GET</code> / <code>do_POST</code> 라우트 테이블(8장) + <code>visualizer/</code> 정적 파일 서빙.

1.3 `focus_autocal.py`와의 관계

`focus_autocal.py`는 하드웨어에 대해 아무것도 모르는 재사용 가능한 순수 알고리즘 라이브러리다 — 카메라/스테이지 콜백을 호출자가 넘겨준다. `FocusScanner._run_stage()`가 ESP300 Z-jog와 카메라의 실시간 blob score를 `scan_focus_z()`에 연결해주는 어댑터 역할. Repo 루트의 `run_*.py` standalone 스크립트들은 자기 시리얼/카메라 핸들을 새로 열지 않고 **이미 켜져 있는 `visualizer_server.py`를 HTTP로 조종**한다 — ESP300의 COM 포트는 한 번에 하나만 열 수 있고, Jog 패널이 이미 그 연결을 쥐고 있기 때문(9장).

1.4 세션 상태

도면을 불러오면 `SESSIONS: dict[str, dict]` (모듈 전역, 메모리 상주, 서버 재시작하면 사라짐)에 `uuid.uuid4().hex[:12]` 키로 세션 하나가 생긴다. 이후 `/api/plan`, `/api/export`는 전부 이 `sessionId`를 참조.

02 시작하기 — 서버 실행과 첫 연결

2.1 서버 실행

바탕화면 바로가기 "**DWG to ESP300 열기**" 또는 아래를 직접 실행:

```
C:\Users\bjh14\dwg-to-esp300-lab-code\dwg-esp300-launch.cmd
```

이 배치파일이 하는 일: `netstat` 로 포트 **8768**이 이미 열려있는지 확인 → 없으면 `python`

`visualizer_server.py --host 127.0.0.1 --port 8768` 을 최소화 창으로 새로 띄움 → 브라우저로

`http://127.0.0.1:8768/` 오픈.

포트 주의 - 절대 지키기

`visualizer_server.py` 자체의 argparse 기본 포트는 **8765**다. 8765/8766/8767 세 포트는 이 프로젝트와 무관한 개인 study 서버(sakurai/goldstein/signals)가 영구적으로 쓰는 포트라, 이 프로젝트를 그 위에서 절대 띄우면 안 된다. `dwg-esp300-launch.cmd` 는 올바르게 `--port 8768` 로 덮어써서 실행함 — 반드시 이 launcher로 실행할 것.

같은 repo 루트의 `launch_dwg_to_esp300.ps1` 은 예전 버전 launcher로 `$port = 8766` 이 박혀있던 것을 이번 매뉴얼 작성 중 발견해서 **8768로 고쳐놓음**(2026-09-09). 앞으로도 `.cmd` launcher를 표준으로 쓰고 `.ps1` 은 참고용으로만 둘 것.

2.2 첫 연결 순서

- Jog 탭에서 ESP300 연결.** Port `COM5`, Baud `19200` 이 기본값. **Dry-run** 체크박스를 반드시 해제해야 실제 이동이 일어남 — 체크된 채로는 서버가 `self.dry_run` 이 True라 모든 시리얼 write가 조용히 no-op된다. X-Y range(mm)-Z range(mm)도 이때 지정 — 이게 곧 3장/11장에서 다루는 하드 안전 한도가 된다.
- Focus camera 패널에서 카메라 연결.** Index 기본값 1(`camera_probe.py` 로 먼저 어느 인덱스가 맞는 UVC 장치인지 확인해두면 좋음, 9장). MJPEG 스트림이 뜨고 blob 오버레이(초록=red glow, 노랑=saturated core, 자홍 십자=합성 타겟)가 보이면 정상.
- 도면 로드.** Toolbar에서 파일 선택 또는 경로 입력, "Open path"/"Open file". 샘플이 필요하면 "Sample" 버튼 (`C:\Users\bjh14\Downloads\adjust25design.dwg` 가 있어야 함).
- 초점 스캔.** Autofocus 패널 "Scan focus" — 처음이면 wide+narrow 2단, 이후엔 narrow만.
- 필요하면 "Run on machine".** 7장 참고 — 실행 전 확인 다이얼로그가 뜸.

전원관리 주의

PC가 절전모드로 들어가면 USB-시리얼 연결이 끊기거나 COM 포트 번호가 바뀔 수 있음. 조그/실행 중에는 Windows 절전을 꺼둘 것.

03 화면 구성 — Toolbar & Planner

브라우저 페이지는 2단 그리드: 왼쪽 `workspace` (툴바 + 2D/3D 미리보기 + 타임라인), 오른쪽 `aside.panel` (지표 타일 + 카메라/초점 패널 + Jog/Shapes/Paths 탭).

3.1 Toolbar

컨트롤	기본값	동작
파일 선택 + "Open file"	–	업로드 → <code>POST /api/trace</code> (multipart)
경로 입력 + "Open path"	<code>C:\Users\bjh14\Downloads\adjust25design.dwg</code>	Enter 또는 버튼 → <code>POST /api/trace</code> (JSON)
"Sample"	–	<code>GET /api/default</code> , 없으면 404
Trace 모드	Cut lines only(<code>design</code>)	<code>design</code> 이면 커팅 대상만 자동 선별, <code>all</code> 은 전체 엔티티
Fit longest(mm)	25	최장변 기준 스케일
Scale × / Apply scale	1.0	추가 배율
Rotate(deg)	45	스케일 전 회전
Cut speed(mm/s)	0.4	레이저 on 이동 속도
Travel speed(mm/s)	1.0	레이저 off 이동 속도
Z speed(mm/s)	0.3	초점/디포커스 전환 속도
Optimize path	켜짐	경로 순서 최적화
"Download code"	–	<code>POST /api/export</code> , standalone .py 생성 후 자동 다운로드

속도/스케일 필드를 바꾸면 180ms 디바운스 후 `/api/plan` 이 자동 재요청됨 — 매 키 입력마다 재 계산하지 않음.

3.2 미리보기 영역

`canvas#previewCanvas` 는 라이브러리 없는 순수 Canvas 2D 렌더. `div#stage3d` 는 **Three.js**(`visualizer/vendor/three.module.js` — 로컬 vendor 카피, CDN 아님)로 그린 3D 스테이지 뷰, `OrbitControls` 없이 포인터 드래그로 직접 오빗/휠 줌 구현. HUD에 레이저 상태·현재 Z·세그먼트 번호 표시. 하단 `footer.timeline` 은 재생 트랜스포트 — `requestAnimationFrame` 기반, 타이머 아님.

3.3 지표 타일 & Shapes/Paths 탭

지표 타일 4개: bbox, 경로 수, cut 길이, travel 길이. **Shapes** 탭에서 Line/Rect/Circle/Spiral을 도면 없이 직접 그려 넣을 수 있음("Blank"/"Add shape"/"Delete selected" → `POST /api/manual`). **Paths** 탭은 개별 경로 체크박스·이름·길이·패스 수를 보여주고 Select All/None, 전체 패스 수 일괄 적용을 지원.

04 화면 구성 — Manual Jog 패널

4.1 연결 필드

Port (기본)

Baud (기본)

X-Y range(mm) 기본 75 - 이 반경을 벗어나는 이동은 서버가 거부(11장)

Z range(mm) 기본 20 - 홈 기준 $\pm$ 이 값을 벗어나면 거부

Dry-run 기본 체크됨(안전). 해제해야 실제 하드웨어가 움직임

4.2 조그 조작

스텝 크기 버튼 8개(0.001~5mm, 기본 활성 0.1mm). 3×3 조그 패드(Y+/X-/STOP/X+/Y-) + 별도 Z focus +/- 행. 포인터를 누르고 있거나 매핑된 키를 누르고 있으면 **120ms마다** `POST /api/jog/move`가 반복 전송됨.

키보드 단축키

키	동작	확인창
Space	즉시 E-STOP	없음 - 바로 실행
↑ / ↓	Y+/Y-	없음 (누르는 동안 120ms 반복)
← / →	X-/X+	없음
PageUp / PageDown	Z focus +/-	없음
Enter (경로 입력창)	"Open path" 실행	-

입력창(INPUT/SELECT/TEXTAREA)에 포커스가 있을 때 이 단축키들이 비활성화됨.

4.3 확인창(confirm) 있는 동작 — 총 6개

동작	확인 문구
Home	"Move stage to machine zero (0, 0, 0)? This can be a large move."
Set local home	목표 X/Y/Z와 현재 위치를 보여주는 동적 문구 + "No motion, no change to the real machine home..."
Go to local home	"Move stage to the labeled local home position?"
Go-to X/Y/Z	"Move stage to X ... / Y ... / Z ... mm?"
Run on machine	"Run the full planned path on the machine? N segments, ~Ns."
Start survey	"Survey N points along the spiral (~M min at this scan speed). Start?"

E-STOP, 단발 조그 이동, 연결/해제, **export**, 경로 삭제, 초점 스캔 버튼은 확인창 없이 즉시 실행됨

— 특히 E-STOP은 의도적으로 무확인 즉시반응(비상정지가 확인창 대기하면 안 되므로).

05 화면 구성 — Focus Camera & Autofocus 패널

5.1 Focus Camera 패널

Index 입력(기본 1) + Connect/Disconnect. `img#cameraStream` 이 `/api/camera/stream` (MJPEG, 15fps)을 그대로 표시. 스트림 위 클릭 = `POST /api/camera/select-target` 로 blob 추적 앵커를 그 위치로 강제 이동(여러 후보 blob 중 특정 것을 지목할 때). 밝기/스팟 수치가 실시간 갱신.

5.2 Autofocus 패널

"Scan focus"(기본) / "Wide rescanning" / "Narrow only" 버튼 → `POST /api/focus/scan`. "Active" 체크박스 → `POST /api/focus/active` — 커두면 X/Y가 움직일 때마다 600ms 디바운스 후 자동으로 narrow 재스캔이 돈다(6장 4절).

실시간 SVG 차트(`svg#focusChart`)가 스캔 곡선을 그리고, 드래그로 범위를 선택하면 그 구간으로 커스텀 스캔을 다시 돌릴 수 있음.

Custom scan 하위 패널

direction(both/down/up) · range(mm) · step(mm) · settle(s) 필드 → `POST /api/focus/scan/custom`. 특정 구간만 다시 훑거나, 속도-신뢰성 트레이드오프를 직접 조정할 때.

Timing check 하위 패널

"Verify timing (slow vs fast)" 버튼 → `POST /api/focus/latency-check`. 아주 느리고 여유로운 기준 스캔과 실제 production 타이밍 스캔을 같은 창에서 두 번 돌려 피크 위치 차이(`deltaMm`)와 그걸 "지연 시간"으로 환산한 `impliedLagS`를 보여줌 — 6장 5절에서 이 지표의 한계를 다룸.

Spiral survey 하위 패널

narrow-only 토글 + "Survey spiral path"/"Stop survey" → `POST /api/focus/survey/start|stop`. X/Y 웨이포인트를 순서대로 돌며 각 지점에서 초점 스캔 → (x,y)→Z 오프셋/밝기 맵을 만듦. SVG로 진행 상황 표시.

SURVEY 실행 전 확인

2026-09-09 세션에서 survey 도중 카메라 프리즈 버그로 한 waypoint에서 20분 넘게 멈춘 사고가 있었음(10장/11장). 지금은 watchdog이 코드에 들어가 있지만 아직 실기계 재검증 전 — survey를 돌릴 땐 옆에서 지켜보고, 이상하면 `Space`로 즉시 E-STOP.

06 자동초점 알고리즘 상세

`focus_autocal.py` 가 순수 알고리즘, `visualizer_server.py` 의 `FocusScanner` 가 이 리그 하드웨어에 맞춘 어댑터.

6.1 `scan_focus_z()` — 핵심 스캔 함수

```
scan_focus_z(move_z, fire_pulse, read_brightness, center_z,
             range_mm=0.3, step_mm=0.02, settle_s=0.05,
             direction="both", adaptive_rise_ratio=1.5,
             adaptive_subdivisions=4) -> FocusScanResult
```

- coarse sweep.** `center_z` 기준 `direction` 에 따라 오프셋 목록 생성 — `both` 는 $\pm$ range, `down` 은 0..-range, `up` 은 0..+range.
- adaptive subdivision.** 인접 샘플 쌍의 밝기 비율이 `adaptive_rise_ratio` (1.5)를 넘으면 "spike"로 판단해서 그 구간에 `adaptive_subdivisions` (4)개를 더 촘촘히 찍음 — 빠른 coarse 스캔으로도 좁은 실제 피크를 놓치지 않게 하는 핵심 트릭.
- 피크 선택.** coarse+refined 샘플 전체에서 최댓값 위치를 찾음.
- 파라볼릭 피팅.** 그 피크가 양쪽에 이웃을 갖고 있으면 세 점을 지나는 포물선의 꼭짓점을 계산해 `best_z` 로 사용(등간격 아니어도 정확한 공식) — 서브스텝 해상도. 아니면 원samples 중 최댓값 위치 그대로 사용.

노이즈에 강건함이 필요하면(전체 이웃을 다 보므로) 이 함수를, 속도가 우선이고 곡선이 매끈하다고 확신하면 아래 `hill_climb_focus_z()` 를 쓴다.

6.2 다른 두 진입점

함수	용도
<code>calibrate_surface()</code>	표면 하나당 한 번 호출하는 고수준 진입점. ROI를 <code>baseline/pulse diff</code> 로 자동 재탐지한 뒤 <code>scan_focus_z</code> 에 위임. 카메라 각도가 바뀌어도 다시 호출만 하면 스스로 재조정.
<code>hill_climb_focus_z()</code>	양방향 언덕오르기. 더 밝으면 같은 방향·같은 스텝 유지, 아니면 방향 반전+스텝 절반. <code>fit_used</code> 는 항상 <code>False</code> . 6장 8절 PID 검증 결과와 같은 이유로, 잔여 노이즈가 있는 지금 곡선에서는 국소 최댓값에 갇히기 쉬움 — 곡선이 이미 매끈하다고 알려진 상황(작은 알려진 drift 재보정)에서만 권장.

`FocusScanner._run_stage()` 는 `scan_focus_z` 를 직접 부른다 — `calibrate_surface` 나 `focus_autocal` 의 `BrightnessCamera` /ROI 자동탐지는 안 쓴다. 실카메라의 blob 추적(`locate_bright_regions`)이 이미 서버 쪽에 있기 때문.

6.3 밝기 스코어 — `locate_bright_regions()`

`visualizer_server.py` 에만 있는 함수(`focus_autocal.py` 엔 없음). 프레임에서 red-hue 블롭과 saturated-core(회색값 ≥ 250) 블롭을 각각 찾고, core가 안쪽에 들어있는 red 블롭을 "진짜 타깃 후보" 페어로 취급. 후보가 여러이면 이전 프레임에서 골랐던 위치(`prior_xy`)에 가장 가까운 것으로 연속성 유지. score는 그 페어 마스크 위 픽셀 값의 합.

6.4 2026-09-09 수정 — 왜 곡선이 들쭉날쭉했나

근본 원인

페어(red+core)를 못 찾으면 score가 정확히 0.0으로 떨어졌음 — 연속 감쇠가 아니라 이진 on/off. 실측: $\pm 1.0\text{mm}/0.01\text{mm}$ 스캔 417샘플 중 142개(34.1%)가 정확히 0.0.

수정 두 가지, 둘 다 `visualizer_server.py` :

- `FOCUS_BRIGHTNESS_SAMPLES` 2→5, 평균을 mean에서 **median**으로. 홀수 5장 중 최대 2장이 드롭아웃 나도 안 흔들림.
- `locate_bright_regions()` 가 페어를 못 찾을 때, 0.0 대신 **core 없이 가장 그럴듯한 red 블롭 단독 score**로 대체. 페어 score는 core 픽셀까지 더하므로 항상 이 fallback보다 크고, 오탐 없이 곡선이 완만히 이어짐.

검증: median-of-5 단독으로 0% 못 만듦(초점 먼 구간은 애초에 절반 넘게 진짜 드롭아웃이라 median도 0 반환) — fallback을 더해야 0.0이 완전히 사라지고 total variation/peak가 18.18→15.22로 줄어듦.

6.5 PID 제어기법 타당성 검증

실측 곡선을 보간해 두 컨트롤러를 오프라인 시뮬레이션함(2026-09-09):

방식	시작점	결과
Naive brightness-PID error = target - 측정 밝기	$z_0 = -0.40$	+0.575mm (참값에서 +0.275mm 오차)
	$z_0 = +0.90$	+18.5mm로 폭주
Slope-seeking P-control error = 국소 기울기	$z_0 = -0.40$	-0.58mm (잔여 스파이크에 걸림)
	$z_0 = +0.90$	+0.855mm (잔여 스파이크에 걸림)

결론: PID는 이 문제 구조에 안 맞음. 밝기-오차 PID는 피크 좌우 어느 쪽에 있는지에 따라 필요한 이동 방향의 부호가 반대인데, 고정 부호 계인은 이걸 구분 못 해 피크를 지나쳐 있으면 오차를 계속 같은 방향으로 키우며 폭주함. 부호가 명확한 slope-seeking조차 6.3절의 잔여 스파이크(9장)에 걸려 국소 최댓값에 갇힘. 지금 쓰는 전체 그리드 스캔 + 파라볼릭 피팅이 이 노이즈 특성에 더 강건함.

07 가공 실행 — Run on Machine

7.1 흐름

Jog 탭 안 "Run planned path on machine" 버튼(`#jogRunButton`) → 확인창 → `POST /api/run/start` (현재 plan의 `segments` 전달) → `JOG.run_plan()` 이 백그라운드 스레드 (`_run_worker`) 시작 → 프론트가 `GET /api/run/status` 를 **200ms**마다 폴링해서 진행률 표시.
"Stop"(`#jogRunStopButton`) → `POST /api/run/stop`.

7.2 세그먼트 실행 방식

세그먼트마다: 목표까지 걸릴 시간(`duration`)으로 축별 속도를 역산 → `nVA` (속도 설정) → `nPA` (절대 위치 이동) 전송 → 도착 확인 폴링. 타임아웃은 `max(3.0, duration*4 + 2.0)` 초, 허용오차 0.02mm. 타임아웃 안에 못 들어오면 **그 자리에서 즉시 예외** — 예전엔 타임아웃을 도착으로 오인하고 계속 진행했었는데, 그게 축 stall/fault를 놓치는 원인이라 지금은 바로 멈추고 `_fault_context()` (ESP300 에러 버퍼/축 에러 상태)를 붙여서 보고함.

7.3 "이유 없이 끊기는" 문제 — 2026-09-09 수정

원인 두 가지, 둘 다 코드로 고침(실기계 재검증 전):

- 중복 폴링.** 백그라운드 `_poll_loop` (위치 갱신용 heartbeat)가 run 중에도 계속 같은 축을 `TP` 쿼리하고 있었음 — `_run_worker` 자신도 도착 확인을 위해 같은 축을 폴링하므로 트래픽이 사실상 두 배. 수정: run이 진행 중이면 `_poll_loop` 는 자기 폴링을 켜.
- 일회성 통신 오류 무재시도.** `_read_axis_position()` 이 진짜 `serial.SerialException` 을 만나면 재시도 없이 즉시 전체 연결을 끊었음(응답 형식이 이상할 때만 재시도하고 있었음). 수정: 같은 재시도 예산(`JOG_TP_RETRIES` =5회) 안에서 통신 오류도 같이 재시도.

"입력초과"는 코드상 매 구간 도착 확인 후에만 다음 구간으로 넘어가는 구조라 무한정 쌓이는 종류는 아니었음.

08 HTTP API 레퍼런스

모든 라우트는 `visualizer_server.py` 의 `do_GET` / `do_POST` 안 단순 if/elif 체인(프레임워크 없음).

모든 JSON 응답은 `Content-Type: application/json; charset=utf-8`, `Cache-Control: no-store`. `do_POST` 는 전체를 `try/except Exception` 로 감싸서 어떤 예외든 `400 + {"error": str(exc)}` 로 변환함 — 스크립트에서 에러 처리할 때 이 형태를 그대로 기대하면 됨.

8.1 GET

경로	설명
<code>/api/motion?script=full outline</code>	사전 생성된 스크립트(<code>adjust25design_esp300.py</code> 또는 <code>_outline_</code>)의 <code>build_segments()</code> 결과 반환
<code>/api/default</code>	<code>DEFAULT_SOURCE</code> 트레이스, 없으면 <code>404</code>
<code>/api/session?id=</code>	세션 dict, 없으면 <code>404</code>
<code>/api/download?file=</code>	<code>generated/</code> 밑 <code>.py</code> 다운로드(경로탈출 방어, <code>.py</code> 만 허용)
<code>/api/jog/status</code>	<code>JOG.status()</code>
<code>/api/run/status</code>	<code>JOG.run_status()</code>
<code>/api/camera/status</code>	<code>CAMERA.status()</code>
<code>/api/camera/stream</code>	MJPEG, 15fps, boundary <code>focusframe</code>
<code>/api/focus/status</code>	<code>FOCUS.status()</code>
<code>/api/focus/log?limit=200</code>	<code>focus_scan_log.csv</code> 마지막 N행, dict 리스트
그 외	<code>visualizer/</code> 정적 파일(빈 경로→ <code>index.html</code>)

8.2 POST — Planning

경로	주요 필드	설명
<code>/api/trace</code>	<code>sourcePath</code> /multipart, <code>traceMode</code> , <code>fitLongestMm</code> (25), <code>flatnessMm</code> (0.02), <code>feedRate</code> (0.4), <code>travelRate</code> (1.0), <code>rotationDeg</code> (45)...	DXF 파싱→선별→회전→스케일→변환, 새 세션 생성
<code>/api/manual</code>	<code>name</code> , <code>feedRate</code> , <code>travelRate</code> , <code>paths[]</code>	Shapes 탭에서 그린 도형으로 세션 생성(<code>source_type="manual"</code>)
<code>/api/plan</code>	<code>sessionId</code> , <code>paths</code> , <code>optimize</code> , <code>feedRate</code> , <code>travelRate</code> , <code>focusZMm</code> , <code>zDefocusMm</code> , <code>zSpeedMmS</code>	<code>build_plan()</code> → <code>segments</code> 포함 전체 계획 반환
<code>/api/export</code>	plan 필드 + <code>port</code> (COM5), <code>baudrate</code> (19200)	<code>standalone.py</code> + SVG 미리보기 생성, 다운로드 URL 반환

8.3 POST — Jog / Run

경로	주요 필드	설명
/api/jog/connect	port (COM5), baudrate (19200), dryRun (true), rangeXYmm, rangeZmm	연결 + 안전 한도 설정
/api/jog/disconnect	-	연결 해제
/api/jog/move	axis, deltaMm (0), speedMmS (0.5)	상대 조그 1회
/api/jog/estop	-	즉시 정지, 연결 강제 해제 (11장)
/api/jog/diag	-	ESP300 상태 읽기 전용 진단 (TB?, 축별 MO? / MD? / TE? / ID?)
/api/jog/home	-	로컬 (0,0,0)으로 이동
/api/jog/go-local-home	-	지정된 로컬 홈으로 이동
/api/jog/set-local-home	x,y,z (0)	소프트웨어 오프셋만 변경, 실제 기계 홈은 안 건드림
/api/jog/goto	x,y,z (0)	절대좌표 이동
/api/run/start	segments[]	계획 전체를 실기계에서 실행 (7장)
/api/run/stop	-	실행 중단

8.4 POST — Camera / Focus

경로	주요 필드	설명
/api/camera/connect	index (1)	UVC 연결 (cv2.CAP_MSMF 고정)
/api/camera/disconnect	-	연결 해제
/api/camera/roi	roi:[x,y,w,h] null	수동 ROI 지정/해제
/api/camera/select-target	x,y	blob 추적 앵커를 클릭 위치로
/api/focus/scan	wide (false), narrowOnly (false)	표준 스캔 실행
/api/focus/active	enabled (false)	XY 이동시 자동 재스캔 감시자 on/off
/api/focus/latency-check	-	slow vs fast 타이밍 자가진단 (6장 5절 관련)
/api/focus/scan/custom	rangeMm (0.1), stepMm (0.005), direction (both), settleS (0.02), centerOffsetMm (0)	파라미터 직접 지정 스캔
/api/focus/survey/start	waypoints:[[x,y]...], narrowOnly (false)	다중 XY 지정 순회 스캔
/api/focus/survey/stop	-	survey 중단

09 Standalone 스크립트 레퍼런스

모두 프로젝트 루트(C:\Users\bjh14\dwg-to-esp300-lab-code\)에서 python <script>.py 로 실행. CLI 인자 없음 — 조정값은 파일 안 상수를 직접 고침. run *.py 4개는 공통으로 BASE_URL = "http://127.0.0.1:8768" 이고, 실행 전 Jog가 live 연결(dry-run 아님) + 카메라 연결이 되어있어야 함(안 되어있으면 RuntimeError).

스크립트	목적	핵심 상수	산출물
camera_probe.py	인덱스 0~4를 cv2.CAP_DSHOW 로 불러 해상도+밝기 출력. 서버/스태이지 불 필요, 장치 인덱스 확인용	MAX_INDEX_TO_SCAN=5	stdout만
run_focus_zscan.py	coarse→fine 2단 Z 스캔, fine은 coarse 피크에 자동 중심	COARSE_RANGE_MM=0.8 , COARSE_STEP_MM=0.03 , FINE_RANGE_MM=0.05 , FINE_STEP_MM=0.002	stdout
run_local_fine_scan.py	현재 위치 기준 양방향 정밀 스캔, 끝나면 원위치 복귀	RANGE_MM=2.0 , STEP_MM=0.01	local_fine_scan.png
run_zaxis_diagnostic.py	0→10mm 전진 스윕, 명령한 이동량과 실제 focusTrimMm 일치 검증. 복귀 없음	STEP_MM=0.1 , MAX_TOTAL_MM=10.0	zaxis_diagnostic.png (2단)
run_descending_search.py	한쪽 방향으로만 내려가며 rise-then-fall 패턴 검출되면 즉시 정지. /api/camera/status 의 raw mean 직접 사용(blob score 아님)	PEAK_RISE_RATIO=1.4 , PEAK_FALL_FRACTION=0.7	descending_search.png
test_scan_quality_metrics.py	assert 기반 self-check(pytest 아님). python test_scan_quality_metrics.py 로 실행, "OK" 출력되면 통과	-	stdout "OK"
gen_check.py	서버/focus_autocal 의존 없는 독립 실행형 경로 러너, 자체 pyserial 직결. CLI 인자 있음	--port COM5 --baudrate 19200 --passes N --dry-run --live 등	stdout

GEN_CHECK.PY 주의

--dry-run 은 store_true 인데 기본값이 이미 True임 — 별도 --live 를 안 주면 아무것도 실제로 움직이지 않는다. 실기계로 돌리려면 --live 를 명시적으로 붙일 것.

10 자동 수집 데이터 & 로그

전부 `_append_csv_row(path, header, row)`를 통해 기록됨 — 파일이 없을 때만 헤더를 쓰고, 디스크 에러는 조용히 무시(로깅이 실제 동작을 절대 방해하면 안 되므로). 5개 모두 **서버가 알아서 씬, 수동 스텝 없음**.

파일	컬럼	기록 시점
<code>focus_scan_log.csv</code>	<code>timestamp_utc, x_mm, y_mm, z_mm, best_offset_mm, fit_used, peak_blob_score, used_wide, plane_samples</code>	스캔 완료마다 1행
<code>focus_scan_quality_log.csv</code>	<code>timestamp_utc, x_mm, y_mm, z_mm, label, n_samples, zero_dropouts, zero_dropout_pct, roughness_tv_over_peak</code>	스캔 완료마다, narrow/primary 곡선만 대상
<code>camera_freeze_log.csv</code>	<code>timestamp_utc, frozen_frame_count, stale_mean, stale_max</code>	프리즈 watchdog 발동 시 (11장)
<code>serial_retry_log.csv</code>	<code>timestamp_utc, axis, comm_error_attempts, outcome (recovered / failed)</code>	통신 오류 겪을 때마다
<code>focus_photo_log.csv</code> + <code>focus_photos/</code>	<code>timestamp_utc, filename, x_mm, y_mm, z_mm, best_offset_mm, peak_blob_score, label</code> / 파일명 <code>%Y%m%dT%H%M%S_Z_offset±N.NNNNmm.jpg</code>	스캔이 검출된 초점 위치로 스테이지를 이동 완료할 때마다 blob 오버레이 포함 프레임 저장

`GET /api/focus/log?limit=200` 은 `focus_scan_log.csv` 만 조회 가능(다른 4개는 아직 조회 API 없음 — 필요해지면 같은 패턴으로 추가 가능).

과거 산출물

같은 폴더에 `focus_scan_log_meanmetric_archive.csv` (구버전, `peak_brightness` 컬럼명 사용) 와 `z_micron_scan_data.csv` 가 남아있음 — 지금 시스템이 자동으로 쓰는 파일이 아니라 과거 standalone 스크립트 실행 결과 아카이브.

11 안전장치 전체 목록

#	메커니즘	동작
1	Z range 캡	<code>z_range</code> (기본 <code>JOG_Z_RANGE_MM=10</code> , 연결시 <code>rangeZmm</code> 로 재지정 가능). <code>move()</code> / <code>goto()</code> 둘 다에서 <code>abs(new_raw - z_home_raw) > z_range</code> 면 실제 시리얼 명령을 보내기 전에 <code>RuntimeError("Move blocked: outside Z safety range.")</code> .
2	XY 반경 캡	<code>limit_center</code> / <code>limit_radius</code> (세션의 <code>usable_center_xy_mm</code> / <code>usable_diameter_mm</code> 에서 유도, 없으면 중심 <code>[12.5,12.5]</code> 반지름 <code>12.5</code>). 반경 밖 이동은 <code>"Move blocked: outside usable glass circle."</code> 로 사전 차단.
3	E-STOP	<code>estop()</code> 이 진행중인 <code>run</code> 을 멈추고, 모든 축에 <code>AB (abort)+MF (motor off)</code> 전송, <code>connected=False</code> 로 강제 전환 – 재연결해야 다시 조그 가능. <code>/api/jog/estop</code> , <code>STOP</code> 패드, <code>Space</code> 키 셋 다 확인창 없이 즉시.
4	Dry-run 기본값	연결시 기본 <code>True</code> . <code>_write()</code> / <code>_query()</code> 가 <code>dry-run</code> 이면 즉시 <code>no-op</code> , 시리얼 바이트 자체가 안 나감. 초점 스캔도 <code>dry-run</code> 이면 아예 시작을 거부.
5	Run 중 도착 타임아웃	세그먼트당 <code>max(3.0, duration*4+2.0)</code> 초 안에 <code>0.02mm</code> 오차로 도착 확인 안 되면 즉시 예외 – 축 <code>stall/fault</code> 를 놓치지 않음(7장).
6	카메라 프리즈 watchdog 2026-09-09	연속 프레임의 <code>(mean,max)</code> 시그니처가 <code>CAMERA_STREAM_FPS*5</code> (75프레임, ~5초)만큼 그대로 반복되면 진짜 연결 끊김과 동일하게 <code>disconnect</code> – UVC 드라이버가 <code>ok=True</code> 를 주면서 같은 프레임을 무한 반환하는 결함 대응.
7	시리얼 재시도-후-차단 2026-09-09	<code>SerialCommError</code> 를 파싱 오류와 같은 재시도 예산(<code>JOG_TP_RETRIES=5</code>)에 포함 – 일회성 통신 hiccup으로 즉시 끊기지 않음. 5회 다 실패해야 진짜 끊긴 것으로 판단.
8	Run 중 폴링 일시정지 2026-09-09	<code>_run_state["running"]</code> 이 <code>True</code> 면 <code>_poll_loop</code> 가 자기 폴링을 쉼 – 중복 트래픽으로 인한 통신 오류 노출 시간 감소.

6·7·8번은 아직 실기계 미검증

2026-09-09 세션에서 코드로 작성/문법 확인만 마침. 다음 라이브 세션에서 서버 재시작 후 실제 하드웨어로 재검증 필요(카메라 재연결 시나리오, run 장시간 실행 시나리오 둘 다).

보류된 항목 – 렌즈-세라믹 물리 접촉 감지

카메라로 렌즈가 세라믹 시편에 닿기 직전/닿는 순간을 시각적으로 감지하는 기능은 접촉 순간의 참조 영상이 없어 기준을 못 잡고 보류. 필요 시 접촉 예시 영상 확보 후 추가.

A 부록 A — 트러블슈팅

증상 / 에러 메시지	의미	대응
"Move blocked: outside Z safety range."	Z 이동이 <code>z_range</code> 한도를 벗어남 (11장 #1)	의도한 이동이면 <code>rangeZMm</code> 을 늘려서 재연결. 의도치 않았으면 현재 홈 기준을 확인
"Move blocked: outside usable glass circle."	XY 이동이 반경 한도를 벗어남 (11장 #2)	<code>rangeXYMm</code> 재확인, 혹은 세션의 <code>usable circle</code> 설정 확인
"Jog not connected. Connect first."	Jog 미연결 상태에서 이동/스캔 시도	Jog 탭에서 먼저 연결
"Jog is in dry-run mode; connect live to scan focus."	Dry-run 커진 채로 초점 스캔 시도	Dry-run 체크 해제 후 재연결
"Camera not connected."	카메라 미연결 또는 방금 <code>watchdog</code> (11장 #6)에 의해 자동 <code>disconnect</code> 됨	<code>camera_freeze_log.csv</code> 확인 후 재연결
"could not locate spark in baseline/pulse frame diff..." (<code>focus_autocal.py</code>)	ROI 자동탐지 실패 – pulse 세기, 카메라 프레임링, <code>diff_threshold</code> 문제	펄스 파워/카메라 각도 확인, <code>diff_threshold</code> 조정
"Segment N/M timed out waiting for arrival."	Run 중 세그먼트 도착 확인 타임아웃(7장 2절) – 메시지에 <code>ESP300</code> 에러 버퍼가 붙어 나옴	불어나온 <code>axis error</code> 코드로 원인 특정, 축 <code>stall/fault</code> 점검
"A scan is already running." / "A survey is in progress."	중복 실행 시도	진행 중인 것부터 끝내거나 중지
스캔 그래프가 0과 큰 값 사이를 오가며 톱니 모양	6.4절의 근본 원인(수정 전 증상) – 2026-09-09 이후로는 발생하지 않아야 함	여전히 보이면 <code>median-of-5+fallback</code> 수정이 실제로 반영된 서버가 도는지 확인(서버 재시작 필요했는지)
초점 스캔이 20분 넘게 안 끝남, 값이 한 자리에 고정	카메라 드라이버 프리즈(11장 #6이 잡아야 할 상황)	즉시 <code>Space</code> 로 E-STOP, <code>camera_freeze_log.csv</code> 확인
"Run on machine" 중 갑자기 연결 끊김	7.3절 참고 – 2026-09-09 이전엔 원인 불명이었음	<code>serial_retry_log.csv</code> 에서 <code>failed</code> 행 빈도 확인, 케이블/USB 허브 점검
PID 컨트롤러로 초점 로직을 바꾸고 싶다	6.5절 시뮬레이션 결과, 구조적으로 부적합함이 실측 데이터로 확인됨	바꾸지 말 것 – 대신 잔여 스파이크(9장/11장) 원인 해결에 투자

B 부록 B — 튜닝 상수 전체 표

B.1 Jog

상수	값	의미
JOG_AXES	<code>{"x":1,"y":2,"z":3}</code>	ESP300 축 번호
JOG_DEFAULT_BAUDRATE	19200	기본 시리얼 속도
JOG_MOTOR_SETTLE_S	0.05	연결시 축별 MO 후 대기
JOG_TP_RETRIES	5	TP 위치 읽기 재시도 횟수
JOG_TP_RETRY_DELAY	0.08	재시도 간 대기(초)
JOG_Z_RANGE_MM	10.0	기본 Z 안전 폭(±)
JOG_STEP_MAX_MM	5.0	1회 조그 최대량
JOG_SPEED_MAX_MM_S	10.0	조그 최대 속도
JOG_HOME_SPEED_MM_S	5.0	홈 이동 속도

B.2 Camera

상수	값	의미
CAMERA_STREAM_FPS	15	캡처 루프 / MJPEG 프레임레이트
CAMERA_JPEG_QUALITY	80	JPEG 인코딩 품질
CORE_MIN_GRAY	250	"saturated core" 판정 회색값 임계
BRIGHT_SPOT_MIN_AREA_PX	4	블롭으로 칠 최소 면적(px)
MAX_TRACKED_BLOBS	8	오버레이에 표시할 블롭 최대 개수

B.3 Focus

상수	값	의미
FOCUS_WIDE_RANGE_MM / _STEP_MM	2.0 / 0.01	wide 스캔 범위/스텝
FOCUS_NARROW_RANGE_MM / _STEP_MM	1.0 / 0.01	narrow 스캔 범위/스텝
FOCUS_Z_SPEED_MM_S	0.3	스캔 중 Z 속도
FOCUS_MECH_SETTLE_S	0.02	이동 후 기계적(진동) 정착 대기
FOCUS_EXPOSURE_MARGIN_S	0.05	"fresh frame" 판정에 더하는 여유시간
FOCUS_FRAME_WAIT_TIMEOUT_S	1.0	fresh frame 대기 타임아웃
FOCUS_BRIGHTNESS_SAMPLES	5(2026-09-09 이전 2)	Z 스텝당 median 널 프레임 수
FOCUS_XY_DEBOUNCE_S	0.6	Active 모드 재스캔 디바운스
FOCUS_PLANE_MIN/MAX_SAMPLES	3 / 10	트렌드 평면 피팅 샘플 수 범위
FOCUS_PREDICT_MAX_DELTA_MM	0.1	트렌드 예측이 한 번에 옮길 수 있는 최대 Z
FOCUS_PHOTO_WAIT_TIMEOUT_S	2.0	초점 사진 저장 전 fresh frame 대기 타임아웃

B.4 Planning 기본값

상수	값	의미
DEFAULT_LONGEST_SIDE_MM	25.0	기본 최장변 맞춤 크기
DEFAULT_ROTATION_DEG	45.0	기본 사전 회전
DEFAULT_FEED_RATE_MM_S	0.4	기본 cut 속도
DEFAULT_TRAVEL_RATE_MM_S	1.0	기본 travel 속도
DEFAULT_Z_DEFOCUS_MM	1.0	travel 중 Z 후퇴량
Z_STEP_BACK_PASSES / _MM	10 / 0.001	N패스마다 소재 침식 보정용 Z 추가 후퇴(1μm)
DEFAULT_GLASS_SIZE_MM	25.0 × 25.0	기본 시편 크기

DWG to ESP300 Operator Manual · 생성: 2026-09-09 · 대상 코드: C:\Users\bjh14\dwg-to-esp300-lab-code (2026-09-09 자동초점/안전/로깅 수정 반영) · 이 문서는 코드베이스 전수 조사를 바탕으로 작성됨, 코드가 바뀌면 재생성 필요.